

FA26: DATA-266 Sec 21 & 71 - Generative AI and LLM

Homework 4

Siva Surya Chandran
019130215

What I did

For this assignment I built a decoder-only, GPT-style transformer completely from scratch in PyTorch - no `nn.Transformer`, no `nn.MultiheadAttention`, no HuggingFace models. I wrote the multi-head masked self-attention by hand and stacked it into decoder blocks to build the full model. I trained it on `shakespeare.txt`, which is just the Romeo and Juliet balcony scene (a pretty short text). Everything below - the code, all the outputs, and the write-up for each part - is in the notebook `Mini_GPT_Assignment.ipynb`.

Getting the data ready

The text file has 1,738 characters total, and there are 49 unique characters in it, so that's my vocab size. I tokenized it at the character level (basically just `char_to_idx` / `idx_to_char` dictionaries), then built a sliding window dataset with a sequence length of 128. That gave me 1,609 overlapping input/target pairs, where the target is just the input shifted over by one character - that's the next-character prediction setup.

One thing I want to flag: this text file is really small. With a sequence length of 128 there's only about 1,600 training windows to work with, which isn't much. So I wasn't expecting the model to actually learn general English - I figured it would mostly just memorize pieces of this one passage. I get into this more in the analysis section below.

The model

I built multi-head self-attention manually using `nn.Linear` layers for Q, K, and V, reshaped into separate heads, then did scaled dot-product attention per head with a causal mask (`torch.triu`) so the model can't cheat by looking at future characters. Each decoder block does attention with a residual connection and a `LayerNorm`, then a feedforward network (`Linear` -> `GELU` -> `Linear`) with another residual and `LayerNorm`. I used the pre-norm setup (`LayerNorm` before each sub-layer) since it tends to train more smoothly once you stack multiple blocks.

Setting	Value
Hidden dimension (<code>d_model</code>)	128
Number of attention heads	4
Number of decoder blocks	4
Sequence length	128
Total parameters	822,321

Training

I trained with Adam (learning rate $3e-4$), batch size 16, for 8 epochs, using cross entropy loss on the next-character predictions. The loss went down steadily the whole time and never bounced back up, which is what I wanted to see:

Epoch	1	2	3	4	5	6	7	8
Loss	2.74	2.24	2.11	1.98	1.79	1.55	1.29	1.05

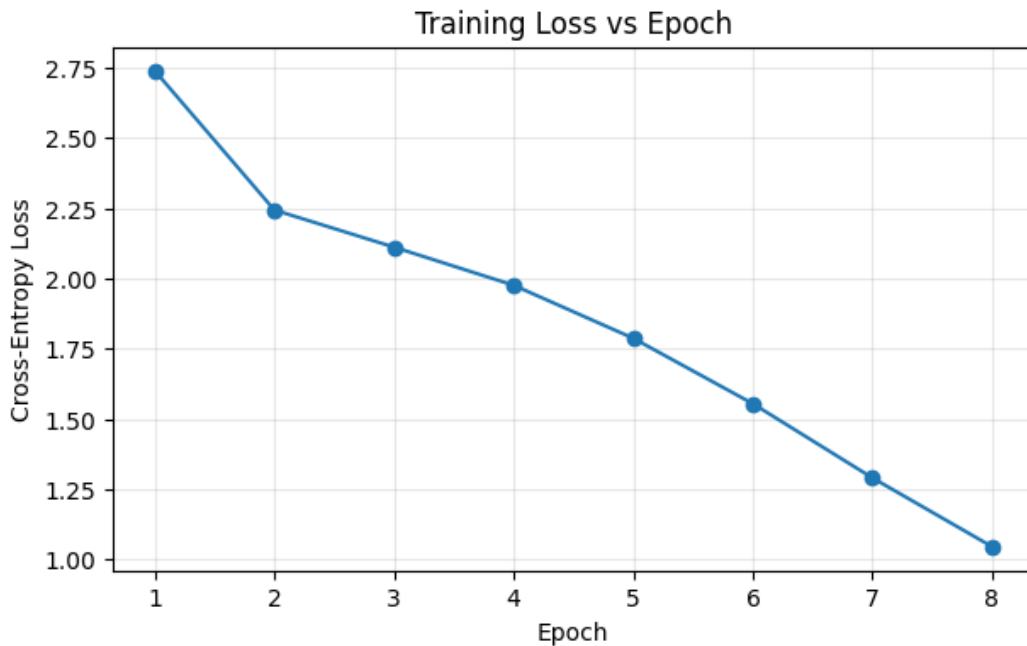


Figure 1: training loss over the 8 epochs

Sample outputs from each decoding method

Prompt I used for all of these: "ROMEO:"

Greedy decoding:

ROMEO:

I takee thy word word:

Call me ne be wet;

JULIET:

What that thu thy namat thus Moueld, not, not,

And, nor nor nor nor nor nor bear be ny or or or or be,

Whorme, ny for orme, far fany or or or or or

Temperature sampling (T = 0.5, 1.0, 1.5):

T=0.5: ROMEO:

I take thy name? ame, nor d whar she saind pakll se seas
Rotakers thers aks ays no wother wof would,

T=1.0: ROMEO:

By na name;
Thar ort thain f thar mar fue.
Shal se s as l ak aks:

T=1.5: ROMEO:

W:
DRomeo? Take NoreoULIEO Romeo! wher tho:
:
JULeoULIEs'JULIETw y narell thy anare ean,

Top-k sampling (k = 3, k = 15):

k=3: ROMEO:

I tall thy ame

I name, nor ar ar tondell be she s

k=15: ROMEO:

Whar at that more, shy not tat nat hor telongh heouser n no name?

Mothe, r nor nd norme par Carthele

JULIETJULIET:

O Romeouthy whyET:

Comparing the decoding methods

Greedy decoding just grabs the highest probability character every single step. Because of that it's deterministic - same prompt always gives the same output. It tends to be the most "coherent" in the sense that it's always taking the model's best guess, but I noticed it gets stuck in loops pretty easily (you can see it above repeating "or or or or" and "nor nor nor" - once it locks onto a pattern it just keeps going since that pattern keeps looking like the best choice).

Temperature sampling changes how sharp or flat the probability distribution is before sampling from it. At T=0.5 the output looks pretty close to greedy - still somewhat repetitive but a little more varied. At T=1.5 it's a lot more random, you start getting weird character combos and broken words that don't really look like English anymore. So temperature is basically a dial between "safe and repetitive" and "random and messy."

Top-k sampling cuts off the choices to just the top k candidates so it can't pick something really unlikely. With k=3 it stays pretty close to greedy/low temperature behavior. With k=15 there's a lot more room to pick different characters, so the output is more varied than k=3, but it still avoids the really nonsensical stuff you'd get from unrestricted high temperature sampling since the worst options are never even considered.

So which one is most coherent? Greedy decoding, since it's always committing to what the model is most confident about and never taking a risk on a less likely character.

Which one is most diverse? High temperature sampling ($T=1.5$) produces the most different/random output out of everything I tried. If I wanted diversity without it turning into total nonsense, top-k with a larger k (15) seemed like a better tradeoff than raw high temperature.

One big caveat here: shakespeare.txt is a really small file (only about 1,700 characters), so honestly the model is mostly just memorizing bits of this one passage rather than learning general Shakespearean English. That's why you can see fragments like "ROMEO:", "JULIET:", "name", "Romeo" showing up in basically every output regardless of decoding method. With a much bigger dataset I'd expect this same pattern (greedy = safest/most repetitive, high temperature = most diverse/least coherent, top-k = in between) to still hold, but the actual generated text would look a lot more like real English instead of just echoing back memorized pieces of the training text.

Quick summary

Method	Coherence	Diversity	Deterministic?
Greedy	Highest	Lowest	Yes
Temperature ($T=0.5$)	High	Low-ish	No
Temperature ($T=1.5$)	Low	Highest	No
Top-k ($k=3$)	High	Low-ish	No
Top-k ($k=15$)	Medium	High	No